BBM473 - Database Lab. (Spring 2025)

Project Design Report

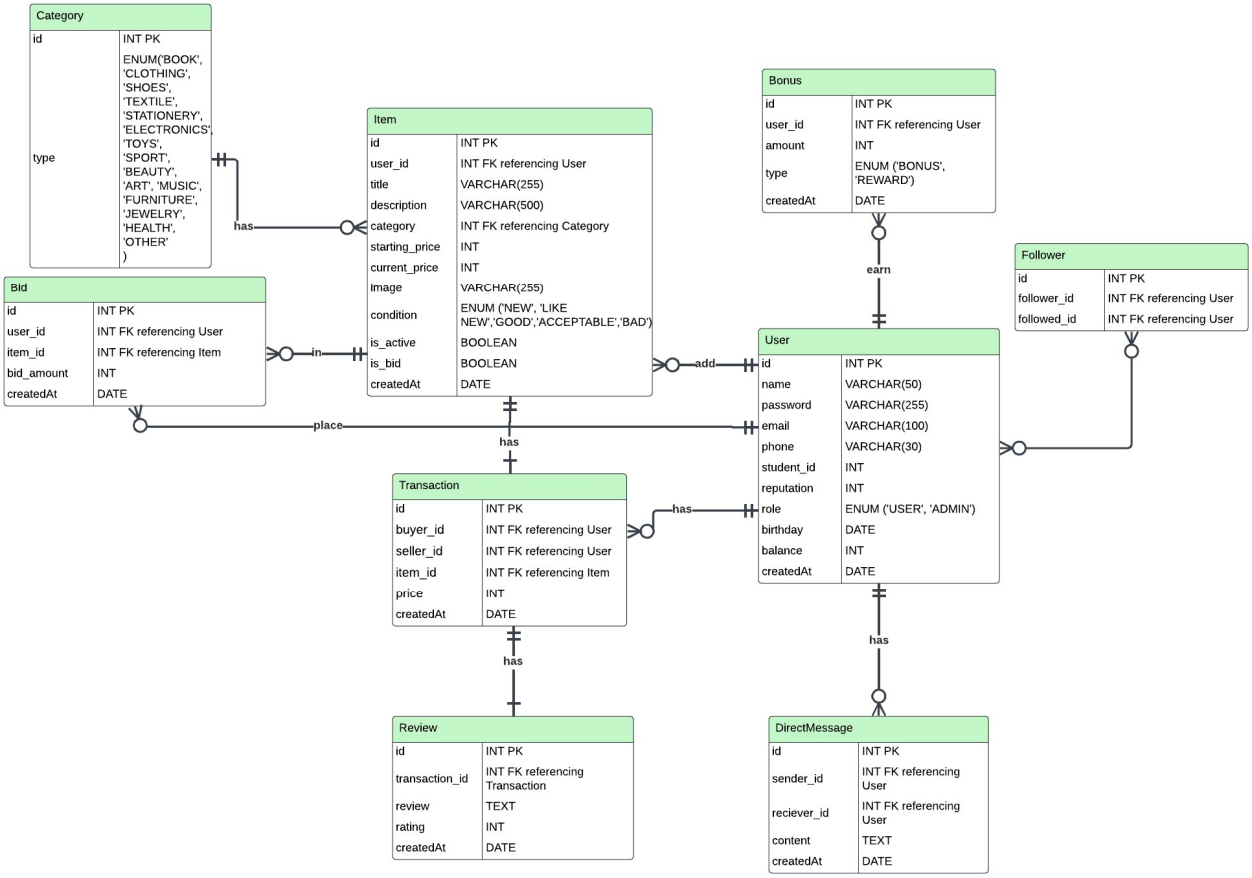
BarterSwap

Student Name: Gülvera Yazılıtaş

Student ID: 2210356111

Department of Computer Engineering
Hacettepe University

1. Entity-Relationship (ER) Diagram



2. Relational Schema and Normalization

User(**id**, name, password, email, phone, student_id, reputation, role, birthday, balance, createdAt)

Item(**id**, user_id, title, description, category, starting_price, current_price, image, condition, is_active ,is_bid, createdAt)

Transaction(**id**, buyer_id, seller_id, item_id, price, createdAt)

Bid(**id**, user_id, item_id, bid_amount, createdAt)

Bonus(**id**, user_id, amount, type, createdAt)

DirectMessage(**id**, sender_id, reciever_id, content, createdAt)

Review(**id**, transaction_id, review, rating, createdAt)
Follower(**id,** follower_id, followed_id)

User
id → name, password, email, phone, student_id, reputation, role, birthday, balance, createdAt
email → id
“id” and “email” are candidate key so it is in BCNF.

Item
id → user_id, title, description, category, starting_price, current_price, image, condition, is_active, is_bid, createdAt
“id” is candidate key so it is in BCNF.

Transaction
id → buyer_id, seller_id, item_id, price, createdAt
item_id → seller_id
“id” is candidate however item_id is not. So it violates BCNF but it is still in 3NF.

Bid
id → user_id, item_id, bid_amount, createdAt
“id” is candidate key so it is in BCNF.

Bonus
id → user_id, amount, type, createdAt
“id” is candidate key so it is in BCNF.

DirectMessage
id → sender_id, receiver_id, content, createdAt
“id” is candidate key so it is in BCNF.

Review
id → transaction_id, review, rating, createdAt
transaction_id → id,review, rating, createdAt
“id” and “transaction_id” are candidate key so it is in BCNF.

Follower
id → follower_id, followed_id
“id” is candidate key so it is in BCNF.

3. Data Definition Language (DDL) Statements

USER

```
-- Type: enum_users_role
-- DROP TYPE IF EXISTS enum_users_role;
CREATE TYPE enum_users_role AS ENUM
    ('USER', 'ADMIN');

-- Table: users
-- DROP TABLE IF EXISTS users;
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    password VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    student_id VARCHAR(255) NOT NULL,
    reputation INTEGER NOT NULL,
    role enum_users_role NOT NULL DEFAULT 'USER',
    phone VARCHAR(255),
    birthday DATE NOT NULL,
    balance INTEGER NOT NULL
);
```

ITEM

```
-- Type: enum_items_condition
-- DROP TYPE IF EXISTS enum_items_condition;
CREATE TYPE enum_items_condition AS ENUM
    ('NEW', 'LIKE NEW', 'GOOD', 'ACCEPTABLE', 'BAD');

CREATE TABLE IF NOT EXISTS items (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    title VARCHAR(255) NOT NULL,
    description VARCHAR(500),
    category INTEGER NOT NULL,
    starting_price INTEGER NOT NULL,
    current_price INTEGER,
    image VARCHAR(255),
    condition enum_items_condition NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    is_bid BOOLEAN DEFAULT FALSE,
    createdAt TIMESTAMPTZ NOT NULL,

    FOREIGN KEY (user_id) REFERENCES users(id)
        ON UPDATE CASCADE ON DELETE CASCADE,
```

```
    FOREIGN KEY (category) REFERENCES categories(id)
    ON UPDATE CASCADE ON DELETE CASCADE
);
```

TRANSACTION

```
CREATE TABLE IF NOT EXISTS transactions (
  id SERIAL PRIMARY KEY,
  buyer_id INTEGER NOT NULL,
  seller_id INTEGER NOT NULL,
  item_id INTEGER NOT NULL,
  price INTEGER NOT NULL,
  createdAt TIMESTAMPTZ NOT NULL,
  FOREIGN KEY (buyer_id) REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (seller_id) REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (item_id) REFERENCES items(id) ON UPDATE CASCADE ON DELETE CASCADE
);
```

BID

```
CREATE TABLE IF NOT EXISTS bids (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL,
  item_id INTEGER NOT NULL,
  bid_amount INTEGER NOT NULL,
  createdAt TIMESTAMPTZ NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (item_id) REFERENCES items(id) ON UPDATE CASCADE ON DELETE CASCADE
);
```

BONUS

```
-- Type: enum_bonuses_type
-- DROP TYPE IF EXISTS enum_bonuses_type;
CREATE TYPE enum_bonuses_type AS ENUM
  ('BONUS', 'REWARD');

-- Table: bonuses
-- DROP TABLE IF EXISTS bonuses;
CREATE TABLE IF NOT EXISTS bonuses (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL,
  amount INTEGER NOT NULL,
  type ENUM('BONUS', 'REWARD') NOT NULL,
  createdAt TIMESTAMPTZ NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE
);
```

DIRECT MESSAGE

```
CREATE TABLE IF NOT EXISTS direct_messages (
  id SERIAL PRIMARY KEY,
  sender_id INTEGER NOT NULL,
  receiver_id INTEGER NOT NULL,
  content TEXT,
  createdAt TIMESTAMPTZ NOT NULL,
  FOREIGN KEY (sender_id) REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (receiver_id) REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE
);
```

REVIEW

```
CREATE TABLE IF NOT EXISTS reviews (
  id SERIAL PRIMARY KEY,
  transaction_id INTEGER NOT NULL,
  review TEXT,
  rating INTEGER NOT NULL,
  createdAt TIMESTAMPTZ NOT NULL,
  FOREIGN KEY (transaction_id) REFERENCES transactions(id) ON UPDATE CASCADE ON DELETE CASCADE
);
```

FOLLOWER

```
CREATE TABLE IF NOT EXISTS followers (
  id SERIAL PRIMARY KEY,
  follower_id INTEGER NOT NULL,
  followed_id INTEGER NOT NULL,
  FOREIGN KEY (follower_id) REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (followed_id) REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE
);
```

4. SQL Transaction Implementations

Purchase Function

```
CREATE OR REPLACE FUNCTION purchase_item(buyer_id INT, item_id INT)
RETURNS TEXT AS $$
DECLARE
  seller_id INT;
  price INT;
BEGIN
  -- Get info
  SELECT user_id, current_price INTO seller_id, price
  FROM items
  WHERE id = item_id FOR UPDATE;

  -- Balance control
  IF (SELECT balance FROM users WHERE id = buyer_id) < price THEN
    RAISE EXCEPTION 'Insufficient balance';
  END IF;

  -- Transaction init
  PERFORM pg_advisory_xact_lock(1); -- protects from concurrent transactions

  -- Transaction insert
  INSERT INTO transactions (buyer_id, seller_id, item_id, price, "createdAt")
  VALUES (buyer_id, seller_id, item_id, price, NOW());

  UPDATE users SET balance = balance - price WHERE id = buyer_id;   -- Buyer balance update
  UPDATE users SET balance = balance + price WHERE id = seller_id;   -- Seller balance update

  RETURN 'Purchase successful';

EXCEPTION
  WHEN OTHERS THEN
    RAISE NOTICE 'Transaction failed: %', SQLERRM;
    RAISE;
END;
$$ LANGUAGE plpgsql;
```

This function implements a transactional item purchase process in the auction system. It ensures that the buyer has sufficient balance and then transfers the payment from the buyer to the seller securely and atomically.

The function first retrieves the seller's ID and current price of the item using SELECT ... FOR UPDATE to lock the item row and prevent concurrent updates. It verifies whether the buyer has enough balance to complete the purchase. If not, the function raises an exception and aborts the transaction. pg_advisory_xact_lock(1) is used to prevent race conditions by ensuring that only one transaction with this lock runs at a time. A new record is inserted into the transactions table to log the successful transaction with timestamp and details. The buyer's balance is reduced by the item price, and the seller's balance is increased accordingly. If any part of the process fails, the transaction is rolled back, and a notice is raised.

Add Bonus Function

```
CREATE OR REPLACE FUNCTION add_bonus(p_user_id INT, p_amount INT, p_type
enum_bonuses_type)
RETURNS TEXT AS $$
BEGIN
  -- Bonus insert
  INSERT INTO bonuses (user_id, amount, type, "createdAt")
  VALUES (p_user_id, p_amount, p_type, NOW());

  -- add bonus to user
  UPDATE users
  SET balance = balance + p_amount
  WHERE id = p_user_id;

  RETURN 'Bonus successfully added';

EXCEPTION
  WHEN OTHERS THEN
    RAISE NOTICE 'Bonus transaction failed: %', SQLERRM;
    RAISE;
END;
$$ LANGUAGE plpgsql;
```

The function begins by inserting a new record into the bonuses table with the user's ID, the bonus amount, type, and the current timestamp. Immediately after, it updates the corresponding user's balance by adding the bonus amount. If any error occurs during either the insert or update process, the exception block catches it, logs the error message using RAISE NOTICE, and rethrows the exception to ensure the transaction is rolled back automatically. This ensures data integrity and prevents inconsistent updates.

Place Bid Function

```
CREATE OR REPLACE FUNCTION public.place_bid(
    p_user_id INTEGER,
    p_item_id INTEGER,
    p_bid_amount INTEGER
)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    current_price INTEGER;
BEGIN
    -- Lock the item row for update to prevent race conditions
    SELECT current_price INTO current_price
    FROM items
    WHERE id = p_item_id
    FOR UPDATE;

    -- Check if bid is higher than current price
    IF p_bid_amount <= current_price THEN
        RAISE EXCEPTION 'Bid must be higher than current price. Current: %, Bid: %', current_price,
p_bid_amount;
    END IF;

    -- Insert the new bid
    INSERT INTO bids (user_id, item_id, bid_amount, "createdAt")
    VALUES (p_user_id, p_item_id, p_bid_amount, NOW());

    -- Update the item's current price
    UPDATE items
    SET current_price = p_bid_amount
    WHERE id = p_item_id;

    RETURN 'Bid placed successfully';

EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Error placing bid: %', SQLERRM;
        RETURN 'Bid failed';
END;
$$;
```

The place_bid function handles the logic of placing a new bid on an item in an auction-style system. It ensures transactional integrity by locking the target item row and validating that the new bid amount is greater than the current price. If the bid is valid, it inserts the bid into the bids table and updates the current_price field of the item.

5. Database Configuration and Parameters

Database: The system uses **PostgreSQL 17.4** as the database management system. PostgreSQL is chosen for its support for advanced SQL features, robust transaction handling, JSON support, and extensibility through user-defined functions and enums.

Isolation Level: The default transaction isolation level is **Read Committed**, which ensures that any data read is committed at the moment it is read.For critical operations such as purchases or placing a bid, **explicit locking** (e.g., FOR UPDATE, pg_advisory_xact_lock) is used to avoid race conditions and ensure consistency.

Indexing: Primary keys are defined on each table (e.g., users.id, items.id, transactions.id), which automatically creates B-tree indexes. If needed for future performance tuning, custom indexes (e.g., CREATE INDEX ON bids (item_id, bid_amount DESC)) can be added to optimize frequent queries.

Caching: PostgreSQL handles internal caching automatically via shared buffers.

Tuning and Optimization

1 Normalized Schema (BCNF)

Almost all relations in the database schema have been normalized to Boyce-Codd Normal Form. This eliminates redundancy, reduces the risk of update anomalies, and simplifies data integrity constraints.

2 Use of plpgsql Functions:

Business-critical operations like item purchases and placing bids are encapsulated in stored procedures (functions). This brings several advantages:

- Ensures atomicity of multi-step processes (e.g., deducting balance, recording transaction, updating item status).
- Reduces round-trips between the application layer and the database.
- Keeps business logic close to the data, improving maintainability and performance.

3 Concurrency Control:

To handle concurrent access and avoid race conditions, the system uses:

- pg_advisory_xact_lock: a lightweight locking mechanism to coordinate logic that must not overlap (e.g., a single item being purchased at the same time).
- SELECT ... FOR UPDATE: locks specific rows (e.g., item rows during bidding) to prevent conflicting updates during concurrent transactions.

For example, in the place_bid function, the item row is locked using:

```
SELECT current_price INTO current_price
FROM items
WHERE id = p_item_id
FOR UPDATE;
```

4 Time-based Indexing and Auditing:
Most tables include a createdAt field of type timestamp with time zone. This supports:

- Time-based queries (e.g., recent bids, past transactions),
- Auditing and traceability of user actions.

Adding an index on createdAt can improve performance for queries like:

*SELECT * FROM transactions WHERE "createdAt" > NOW() - INTERVAL '7 days';*

6. Query Evaluation Plans

Query 1: Get active items for auction with their highest bid

```
EXPLAIN
SELECT i.id, i.title, i.current_price, MAX(b.bid_amount) AS highest_bid
FROM items i
LEFT JOIN bids b ON i.id = b.item_id
WHERE i.is_active = TRUE AND i.is_bid = TRUE
GROUP BY i.id;
```

1	HashAggregate (cost=42.45..42.57 rows=12 width=528)
2	Group Key: i.id
3	-> Hash Right Join (cost=10.65..40.56 rows=377 width=528)
4	Hash Cond: (b.item_id = i.id)
5	-> Seq Scan on bids b (cost=0.00..25.70 rows=1570 width=8)
6	-> Hash (cost=10.50..10.50 rows=12 width=524)
7	-> Seq Scan on items i (cost=0.00..10.50 rows=12 width=524)
8	Filter: (is_active AND is_bid)

Both items and bids tables are being scanned sequentially (Seq Scan), which means no relevant indexes are being used.
PostgreSQL uses a Hash Right Join between bids and items. This is reasonable given the small size of items, but may be inefficient if either table grows significantly.
The HashAggregate step groups by item ID and computes MAX(bid_amount), which is efficient but could be sped up with indexes.

To reduce execution cost:

- CREATE INDEX idx_bids_item_bid ON bids(item_id, bid_amount DESC);

With this composite index, instead of scanning the entire table, the database engine can directly access matching rows using the index, significantly reducing query time.

Query 2: Get user's transaction history (as buyer)

```
EXPLAIN
SELECT t.id, t.item_id, t.price, t."createdAt"
FROM transactions t
WHERE t.buyer_id = 42
ORDER BY t."createdAt" DESC;
```

```
1  Sort  (cost=28.22..28.24 rows=7 width=20)
2    Sort Key: "createdAt" DESC
3    ->  Seq Scan on transactions t  (cost=0.00..28.12 rows=7 width=20)
4          Filter: (buyer_id = 42)
```

PostgreSQL is scanning the entire transactions table to find rows where buyer_id = 42. This indicates no index is being used on the buyer_id column. The scan filters all rows and keeps only those where the buyer is user 42.
Once matching rows are found, it sorts them by "createdAt" in descending order (most recent first). This introduces additional overhead.

To reduce execution cost:

- CREATE INDEX idx_transactions_buyer_id ON transactions(buyer_id);

Adding this index enables index-based filtering, dramatically improving performance as data volume grows.

- CREATE INDEX idx_transactions_buyer_createdat ON transactions(buyer_id, "createdAt" DESC);

This index allows PostgreSQL to both filter by buyer_id and return the results already sorted by createdAt DESC, which matches the query exactly.

Query 3: Show top 5 users with highest reputation

```
EXPLAIN
SELECT id, name, reputation
FROM users
ORDER BY reputation DESC
LIMIT 5;
```

```
1  Limit  (cost=10.80..10.81 rows=5 width=524)
2    ->  Sort  (cost=10.80..10.87 rows=30 width=524)
3          Sort Key: reputation DESC
4          ->  Seq Scan on users  (cost=0.00..10.30 rows=30 width=524)
```

PostgreSQL is scanning the entire users table to read all rows. The result of the scan is sorted in descending order by reputation. After sorting, only the top 5 rows are returned. This is efficient for small datasets but becomes costly as the number of users grows.

To reduce execution cost:

- CREATE INDEX idx_users_reputation_desc ON users (reputation DESC);

This allows PostgreSQL to avoid a full table scan and sort—it can retrieve the top 5 users directly using the index.

Appendices (Optional)

- https://lucid.app/
- https://www.pgadmin.org/
- https://www.enterprisedb.com/blog/postgresql-query-optimization-performance-tuning-with-explain-analyze